CS-35: Mobile Application Development in Android using Kotlin.

Prepared By : Lathiya Harshal.

Basics of Kotlin

Definition :-

Kotlin is a modern, statically typed programming language developed by **JetBrains** that runs on the **Java Virtual Machine (JVM)** and is officially supported by **Google for Android application development**.

Kotlin is designed to be **concise, safe, and fully interoperable with Java**.

Features of Kotlin

1. **Concise Syntax** – Requires less code compared to Java.
2. **Interoperability** – Kotlin can work with existing Java code.
3. **Null Safety** – Reduces NullPointerException errors.
4. **Object Oriented Language** – Supports classes, inheritance, and interfaces.
5. **Functional Programming Support** – Supports lambda expressions and higher-order functions.
6. **Safe and Reliable** – Strong type checking.
7. **Official Language for Android Development**.

Basic Structure of a Kotlin Program

Syntax Example

```
fun main() {
   println("Hello World")
}
```

Explanation

- `fun` → keyword used to declare a function
- `main()` → entry point of the program
- `println()` → prints output to console

Variables in Kotlin

Variables are declared using **val** and **var**.

Syntax

```
val name = "Harshal"
var age = 20
```

Explanation

Keyword	Meaning
val	Immutable variable (cannot be changed)
var	Mutable variable (can be changed)

Example

```
var number = 10
number = 20
```

Operators in Kotlin

Definition :- Operators are **special symbols used to perform operations on variables and values**.

Types of Operators in Kotlin

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus	a % b

Example

```
var a = 10
var b = 5
println(a + b)
```

---

2. Relational (Comparison) Operators

Used to compare two values.

Operator	Meaning
==	Equal to
! =	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example

```
var a = 10
var b = 20
println(a < b)
```

---

3. Logical Operators

Used to combine conditions.

Operator	Meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

Example

```
var a = 10
var b = 20
println(a < b && b > 5)
```

---

4. Assignment Operators

Used to assign values.

Operator	Example
=	a = 5
+=	a += 5
-=	a -= 5
*=	a *= 5
/=	a /= 5

Example :-

```
var a = 10
a += 5
```

---

5. Unary Operators

Operate on a single operand.

Operator	Meaning
++	Increment
--	Decrement
!	Logical NOT

Example :-

```
var a = 5
a++
```

Operator Priority (Operator Precedence)

Definition

Operator precedence determines the **order in which operations are evaluated in an expression**.

Operators with **higher precedence are executed before lower precedence operators**.

Operator Priority Order (High → Low)

Priority	Operator
1	`++ , --`
2	`* , / , %`
3	`+ , -`
4	`< , > , <= , >=`
5	`== , !=`
6	`&&`
7	`` ` ``
8	`=`

Example

var result = 10 + 5 * 2
println(result)

Execution

5 * 2 = 10
10 + 10 = 20

Output : - 20

Multiplication is executed first because it has **higher precedence than addition**.

Decision Making in Kotlin

Definition

Decision making in Kotlin refers to the process of **executing different blocks of code based on a given condition**. It allows the program to make logical decisions during execution.

Kotlin provides several decision-making statements such as:

- `if`
- `if-else`
- `if-else if-else`
- `when`

These statements evaluate conditions and execute specific code depending on whether the condition is **true or false**.

---

1. if Statement

Definition : The **if statement** is used to execute a block of code **only when a specified condition is true**.

Syntax

```
if(condition){
   // code to execute
}
```

Example

```
fun main() {
   var number = 10

   if(number > 0){
      println("Number is positive")
   }
}
```

Explanation : If the value of the number is greater than `0`, the message **"Number is positive"** will be printed.

---

2. if – else Statement

Definition : The **if–else statement** is used when the program must choose between **two alternatives**.

Syntax

```
if(condition){
   // code if condition is true
}
else{
   // code if condition is false
}
```

Example

```
fun main() {
   var number = 5

   if(number % 2 == 0){
      println("Even Number")
   }
   else{
      println("Odd Number")
   }
}
```

Explanation : If the number is divisible by 2, it prints **Even Number**; otherwise, it prints **Odd Number**.

---

3. if – else if – else Statement

Definition : The **if–else if–else statement** is used when there are **multiple conditions to evaluate**.

Syntax

```
if(condition1)
{// code block 1}
else if(condition2)
{// code block 2}
else{    // code block 3 }
```

Example

```
fun main() {
    var marks = 75

    if(marks >= 90){
        println("Grade A")
    }
    else if(marks >= 60){
        println("Grade B")
    }
    else{
        println("Grade C")
    }
}
```

Explanation : The program checks conditions one by one and executes the block where the condition becomes **true**.

---

4. when Statement

Definition : The **when statement** in Kotlin is used as an alternative to the **switch statement** found in other programming languages. It allows the program to execute a block of code based on **multiple possible values of an expression**.

Syntax

```
when(expression){
    value1 -> statement
    value2 -> statement
    else -> statement
}
```

Example

```
fun main() {
    var day = 3

    when(day){
        1 -> println("Monday")
        2 -> println("Tuesday")
        3 -> println("Wednesday")
        else -> println("Invalid Day")
    }
}
```

Explanation : The value of `day` is checked against different cases, and the corresponding statement is executed.

Advantages of Decision Making

1. Helps in making logical decisions in programs.
2. Allows execution of different code blocks based on conditions.
3. Improves program flexibility and control.

Conclusion

Decision-making statements in Kotlin such as **if, if–else, if–else if–else, and when** allow a program to choose between different actions depending on conditions. These constructs are essential for controlling the flow of a program.

Loop Control in Kotlin

Definition

Loop control statements are used to **execute a block of code repeatedly until a specified condition becomes false**.
Loops reduce code repetition and make programs more efficient.

Kotlin provides the following loops:

- `for` loop
- `while` loop
- `do-while` loop

---

1. for Loop

Definition

The **for loop** is used to iterate through a **range of values, arrays, or collections**.

Syntax

```
for(variable in range){

    // code block

}
```

Example

```
fun main()

{

for(i in 1..5)

{println(i)}
}
```

Output

```
1

2

3

4

5
```

Explanation

`1..5` represents a **range from 1 to 5**, and the loop prints each number.

---

2. while Loop

Definition

The **while loop** executes a block of code **as long as the given condition is true**.

Syntax

```
while(condition){
    // code block
}
```

Example

```
fun main() {
    var i = 1

    while(i <= 5){
        println(i)
        i++
    }
}
```

Explanation

The loop continues until the condition `i <= 5` becomes false.

---

3. do–while Loop

Definition

The **do–while loop** executes the code block **at least once**, even if the condition is false.

Syntax

```
do{
    // code block
}while(condition)
```

Example

```
fun main() {
    var i = 1

    do{
        println(i)
        i++
    }while(i <= 5)
}
```

Explanation

The code runs first and then checks the condition.

Loop Control Statements

Kotlin also provides statements to control loops:

break

Terminates the loop immediately.

Example

```
for(i in 1..5){
    if(i == 3)
        break
    println(i)
}
```

Output

```
1
2
```

---

continue

Skips the current iteration and continues with the next.

Example

```
for(i in 1..5){
    if(i == 3)
        continue
    println(i)
}
```

Output

```
1
2
4
5
```

---

2. Data Structures (Collections) in Kotlin

Definition

Collections in Kotlin are **data structures used to store and manage groups of objects or values**.
They allow programmers to organize, access, and manipulate data efficiently.

Kotlin collections are divided into two types:

1. **Immutable Collections** – cannot be modified after creation
2. **Mutable Collections** – elements can be added, removed, or modified

Types of Collections in Kotlin

1. List : A **List** is an ordered collection that allows **duplicate elements**.

Syntax

```
val numbers = listOf(1, 2, 3, 4)
```

Example

```
fun main() {
    val fruits = listOf("Apple", "Banana", "Mango")


    for(fruit in fruits){
        println(fruit)
    }
}
```

---

2. MutableList : A **MutableList** allows modification of elements.

Syntax

```
val numbers = mutableListOf(1,2,3)
```

Example

```
fun main() {
    val numbers = mutableListOf(10,20,30)
    numbers.add(40)
    println(numbers)
}
```

---

3. Set : A **Set** is a collection that **does not allow duplicate elements**.

Syntax

```
val numbers = setOf(1,2,3)
```

Example

```
fun main() {
    val numbers = setOf(1,2,3,3,4)
    println(numbers)
}
```

Output

```
[1,2,3,4]
```

Duplicates are automatically removed.

---

4. Map: A **Map** stores data in **key-value pairs**.

Syntax

```
val student = mapOf("Name" to "Harshal", "Age" to 20)
```

Example

```
fun main() {
    val student = mapOf("Name" to "Harshal", "Age" to 20)
    println(student["Name"])
}
```

Output : Harshal

Advantages of Collections

1. Efficient storage of multiple values.
2. Easy access and manipulation of data.
3. Provides built-in functions for data management.

Conclusion

Loop control statements (`for`, `while`, `do-while`) allow repeated execution of code based on conditions. Kotlin collections such as **List, Set, and Map** are powerful data structures used to store and manage groups of data efficiently.

Functions in Kotlin

Definition

A **function** in Kotlin is a **block of reusable code designed to perform a specific task**. Functions help in organizing code, reducing repetition, and improving readability.

Functions are declared using the **`fun` keyword**.

Syntax of Function in Kotlin

```
fun functionName(parameters): ReturnType {
    // code block
}
```

Explanation

Part	Description
`fun`	Keyword used to declare a function
`functionName`	Name of the function
`parameters`	Input values passed to the function
`ReturnType`	Type of value returned by the function

Example of a Simple Function

```
fun displayMessage() {
    println("Welcome to Kotlin Programming")
}
fun main() {
    displayMessage()
}
```

Explanation

The function `displayMessage()` prints a message when it is called inside the `main()` function.

Function with Parameters

Definition

A function can accept **input values called parameters**.

Syntax

```
fun functionName(parameter: DataType) { // code }
```

Example

```
fun add(a: Int, b: Int) {
    println(a + b)
}
fun main()
 { add(10, 20) }
```

Output : 30

Explanation

The function `add()` receives two numbers and prints their sum.

Function with Return Value

Definition

A function can return a value using the `return` **keyword**.

Syntax

```
fun functionName(parameters): ReturnType {
    return value
}
```

Example

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
fun main() {
    var result = sum(5, 10)
    println(result)
}
```

Output : 15

Explanation

The function `sum()` returns the addition of two numbers.

Single Expression Function

Kotlin allows writing functions in **short form using a single expression**.

Syntax

```
fun functionName(parameters) = expression
```

Example

```
fun multiply(a: Int, b: Int) = a * b
```

Types of Functions in Kotlin

1. **Function without parameters**
2. **Function with parameters**
3. **Function with return value**
4. **Single expression function**

Advantages of Functions

1. **Code Reusability** – Same function can be used multiple times.
2. **Better Program Structure** – Programs become easier to understand.
3. **Reduces Code Duplication**.
4. **Improves Maintainability**.

Conclusion : Functions in Kotlin are used to perform specific tasks in a program. They are declared using the `fun` keyword and may contain parameters and return values. Functions help in organizing code, improving readability, and enabling code reuse.

Inheritance in Kotlin

Definition

Inheritance is an Object-Oriented Programming (OOP) concept in which **one class (child class) acquires the properties and behavior of another class (parent class)**.

Inheritance promotes **code reusability and hierarchical classification**.

In Kotlin, classes are **final by default**, so to allow inheritance the parent class must be declared using the **open keyword**.

Syntax

```
open class ParentClass {
    // properties and methods
}
class ChildClass : ParentClass() {
    // additional properties and methods
}
```

Example

```
open class Animal {
    fun eat() {
        println("Animal is eating")
    }
}
class Dog : Animal() {
    fun bark() {
        println("Dog is barking")
    }
}
fun main() {
    val d = Dog()
    d.eat()
    d.bark()
}
```

Explanation

- `Animal` is the **parent class**.
- `Dog` is the **child class**.
- The child class inherits the method `eat()` from the parent class.

Advantages of Inheritance

1. Promotes **code reuse**.
2. Reduces redundancy.
3. Represents **real-world relationships**.
4. Improves program structure.

Abstract Class in Kotlin

Definition : An **abstract class** is a class that **cannot be instantiated** and is used as a base class for other classes.

It may contain:

- **Abstract methods** (without implementation)
- **Non-abstract methods** (with implementation)

Abstract classes are declared using the **`abstract` keyword**.

Syntax

```
abstract class ClassName {

    abstract fun methodName()

}
```

Example

```
abstract class Shape {

    abstract fun draw()

    fun display() {

        println("This is a shape")

    }

}

class Circle : Shape() {

    override fun draw() {

        println("Drawing Circle")

    }

}

fun main() {

    val obj = Circle()

    obj.draw()

    obj.display()

}
```

Explanation

- `Shape` is an **abstract class**.
- `draw()` is an **abstract function**.
- `Circle` implements the abstract function.

Key Points

- Cannot create objects of abstract classes.
- Used for **common base functionality**.
- Abstract methods must be **overridden in derived classes**.

Interface in Kotlin

Definition

An **interface** in Kotlin is used to define **a set of abstract methods that a class must implement**.

Interfaces provide **full abstraction** and support **multiple inheritance**.

Interfaces are declared using the **`interface` keyword**.

Syntax

```
interface InterfaceName {
    fun methodName()
}
```

Example

```
interface Vehicle {
    fun start()
}
class Car : Vehicle {
    override fun start() {
        println("Car starts with key")
    }
}
fun main() {
    val obj = Car()
    obj.start()
}
```

Explanation

- `A vehicle` is an interface.
- `Car` implements the interface.
- The function `start()` must be overridden.

Features of Interfaces

1. Supports **multiple inheritance**.
2. Contains **abstract methods**.
3. Methods must be **implemented by the class**.
4. Improves **code flexibility and abstraction**.

`super` Keyword in Kotlin

Definition

The `super` **keyword** is used to **refer to the parent class** and access its **methods, properties, or constructors**.

It is commonly used when a child class overrides a parent method but still wants to use the parent implementation.

Syntax

```
super.methodName()
```

Example

```
open class Animal {
    fun eat() {
        println("Animal is eating")
    }
}
class Dog : Animal() {
    fun show() {
        super.eat()
        println("Dog is eating food")
    }
}
fun main() {
    val obj = Dog()
    obj.show()
}
```

Explanation

- `super.eat()` calls the **parent class method**.
- The child class can extend the behavior.

`This` Keyword in Kotlin

Definition : The `this` **keyword** refers to the **current object of the class**.

It is used to:

- Refer to current class properties
- Call current class methods
- Distinguish between class variables and parameters.

Syntax

this.variableName

Example

```
class Student(val name: String) {
    fun display() {
        println("Student name is " + this.name)
    }
}
fun main() {
    val obj = Student("Harshal")
    obj.display()
}
```

Explanation

- `this.name` refers to the **current object's property**.

Visibility Modifiers in Kotlin

Definition

Visibility modifiers control the **accessibility of classes, functions, and variables**.

Kotlin provides **four visibility modifiers**.

Types of Visibility Modifiers

Modifier	Description
`public`	Accessible everywhere
`private`	Accessible only within the same class
`protected`	Accessible within class and subclasses
`internal`	Accessible within the same module

Example

```
open class Person {
    public var name = "Harshal"
    private var age = 20
    protected var city = "Amreli"
}
```

Explanation

- `public` → accessible from anywhere
- `private` → accessible only within the class
- `protected` → accessible in subclasses
- `internal` → accessible within the same project/module

Conclusion

Object-Oriented Programming in Kotlin provides powerful concepts such as **Inheritance, Abstract Classes, Interfaces, `super`, `this`, and Visibility Modifiers**. These concepts help in building **reusable, maintainable, and well-structured Android applications**.

Unit–2: Introduction to Android & Android Application Design

The Open Handset Alliance (OHA)

Definition

The **Open Handset Alliance (OHA)** is a **consortium of companies formed to develop open standards for mobile devices**.
It was announced in **November 2007** and is led by **Google**.

The alliance works together to **develop and promote the Android platform**.

Members of OHA

The alliance includes companies from different sectors such as:

- Mobile device manufacturers
- Semiconductor companies
- Software companies
- Telecommunication companies

Examples of members:

- Google
- Samsung
- HTC
- LG
- Qualcomm
- Motorola

Objectives of Open Handset Alliance

1. To create **open standards for mobile devices**.
2. To **accelerate innovation in mobile technology**.
3. To provide a **free and open mobile platform**.
4. To improve **mobile user experience**.

Importance of OHA

- It helped launch the **Android operating system**.
- Encourages **open-source development**.
- Promotes **innovation and competition in the mobile industry**.

Conclusion

The **Open Handset Alliance** is a group of technology companies led by Google that collaborates to develop open standards for mobile devices and supports the development of the Android platform.

The Android Platform

Definition

The **Android platform** is an **open-source operating system based on the Linux kernel**, designed mainly for **mobile devices such as smartphones and tablets**.

It provides the **software framework and tools required to build mobile applications**.

Android was developed by **Google** and officially released in **2008**.

Features of Android Platform

1. **Open Source** – Developers can access and modify the source code.
2. **Linux Kernel Based** – Provides security and hardware abstraction.
3. **Multitasking** – Supports running multiple applications simultaneously.
4. **Rich Application Framework** – Provides reusable components.
5. **Connectivity Support** – Supports Wi-Fi, Bluetooth, NFC, and mobile networks.
6. **Media Support** – Supports audio, video, and image formats.
7. **Large Developer Community**.

Components of Android Platform

The Android platform mainly consists of:

1. **Linux Kernel**
2. **Libraries**
3. **Android Runtime**
4. **Application Framework**
5. **Applications**

These components work together to provide the complete Android environment.

Advantages of Android Platform

- Free and open-source
- Supports a wide range of devices
- Large application ecosystem
- Flexible development environment

Conclusion

The Android platform is a powerful open-source mobile operating system that provides a flexible environment for developing and running mobile applications.

Android SDK (Software Development Kit)

Definition

The **Android SDK (Software Development Kit)** is a collection of **development tools used to build, test, and debug Android applications**.

It provides all the necessary **libraries, APIs, and tools required for Android app development**.

Components of Android SDK

1. **Android Studio**
   - Official IDE for Android development.
2. **SDK Tools**
   - Tools used to build Android applications.
3. **Android Emulator**
   - Allows developers to test applications without a physical device.
4. **ADB (Android Debug Bridge)**
   - Used for communication between the computer and Android device.
5. **Libraries and APIs**
   - Predefined code libraries for application development.

Functions of Android SDK

1. Provides tools to **develop Android applications**.
2. Allows developers to **test applications using an emulator**.
3. Helps in **debugging and optimizing applications**.
4. Provides access to Android **APIs and libraries**.

Example Development Process Using SDK

1. Install **Android Studio**.
2. Write application code in **Kotlin or Java**.
3. Design UI using **XML layouts**.
4. Compile and run the application.
5. Test using **Android Emulator or real device**.

Conclusion

The **Android SDK** provides a complete set of tools required for developing Android applications, including libraries, APIs, debugging tools, and testing environments.

Building a Sample Android Application

Definition

Building a **sample Android application** refers to the process of **creating a simple Android app using Android Studio, Kotlin programming language, and Android SDK tools**.
It helps developers understand the **basic structure and workflow of Android application development**.

Requirements for Developing an Android Application

Before building an Android application, the following tools are required:

1. **Android Studio** – Official IDE for Android development
2. **Java Development Kit (JDK)**
3. **Android SDK (Software Development Kit)**
4. **Android Emulator or Physical Android Device**

Steps to Build a Sample Android Application

Step 1: Install Android Studio

Download and install **Android Studio**, which provides all necessary tools for Android development.

Step 2: Create a New Android Project

1. Open **Android Studio**
2. Click **New Project**
3. Select **Empty Activity**
4. Click **Next**
5. Enter the following details:
    - Application Name
    - Package Name
    - Save Location
    - Programming Language (Kotlin)
6. Click **Finish**

Android Studio will automatically generate the project structure.

Android Project Structure

A basic Android project contains the following important files:

1. MainActivity.kt

Contains the **Kotlin code for application logic**.

2. activity_main.xml

Defines the **User Interface (UI) layout**.

3. AndroidManifest.xml

Contains **application configuration, permissions, and activity declarations**.

4. res Folder

Contains application resources such as:

- Layouts
- Images
- Strings
- Colors

Designing the User Interface

The user interface is created using **XML layout files**.

Example layout file:

```
<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:orientation="vertical">
```

```
 <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello Android"/>
</LinearLayout>
```

This layout displays the text **"Hello Android"** on the screen.

Writing Kotlin Code

Example Kotlin code in **MainActivity.kt**:

```
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Explanation

- `onCreate()` → Called when the activity starts
- `setContentView()` → Loads the layout file
- `activity_main` → UI layout defined in XML

Running the Application

Steps to run the app:

1. Connect an **Android device** or start an **Android Emulator**
2. Click the **Run button** in Android Studio
3. The application will be **compiled and installed on the device**
4. The output screen will display the application interface.

Output : The application displays a simple screen showing:

Hello Android

Advantages of Building Sample Applications

1. Helps understand the **Android development process**.
2. Demonstrates the **interaction between UI and Kotlin code**.
3. Useful for **learning Android project structure**.
4. Helps beginners practice **application development concepts**.

Conclusion

Building a sample Android application involves creating a project in Android Studio, designing the user interface using XML, writing application logic using Kotlin, and running the application on an emulator or device. It is the first step in learning Android application development.

Anatomy of an Android Application

Definition

The **anatomy of an Android application** refers to the **basic structure and components that make up an Android application**.
It explains how different files and components work together to create and run an Android app.

An Android application consists of **code files, resource files, configuration files, and application components**.

Main Components of an Android Application

1. AndroidManifest.xml

Definition

The **AndroidManifest.xml** file is the **main configuration file of an Android application**.

It provides essential information about the application to the Android system.

Functions

- Declares **application components** (activities, services, receivers).
- Specifies **permissions** required by the application.
- Defines **application metadata**.
- Specifies the **main entry point of the application**.

Example

```
<manifest>

   <application>

      <activity android:name=".MainActivity">

      </activity>

   </application>

</manifest>
```

---

2. Activities

Definition

An **Activity** represents a **single screen with a user interface in an Android application**.

Each activity performs a specific task and interacts with the user.

Example

Examples of activities include:

- Login screen
- Registration screen
- Home screen

Example Code

```
class MainActivity : AppCompatActivity() {

   override fun onCreate(savedInstanceState: Bundle?) {

      super.onCreate(savedInstanceState)

      setContentView(R.layout.activity_main)}}
```

3. Java/Kotlin Source Files

These files contain the **application logic and functionality** written in **Java or Kotlin**.

Example:

- MainActivity.kt
- LoginActivity.kt

They control the **behavior of the application**.

---

4. Resource Folder (res)

The **res folder** stores all **non-code resources used in the application**.

Examples include:

Folder	Purpose
layout	UI design files (XML)
drawable	Images and icons
values	Strings, colors, styles
menu	Menu layout files

Example layout file:

```
<TextView
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="Hello Android"/>
```

---

5. Assets Folder

The **assets folder** stores raw files such as:

- HTML files
- Fonts
- Media files

These files can be accessed directly by the application.

---

6. Libraries

Libraries provide **predefined functions and APIs** used in Android applications.

Examples:

- Android Support Library
- Google Play Services

Conclusion : The anatomy of an Android application includes components such as **AndroidManifest.xml, activities, source code files, resource folders, assets, and libraries**. These components work together to build and run Android applications.

Android Terminologies

Definition

Android terminologies are important concepts and components used in Android application development.

Important Android Terminologies

1. Activity

Definition

An **Activity** is a **single screen in an Android application that provides a user interface**.

Example:

- Login page
- Settings screen

---

2. Intent

Definition

An **Intent** is a messaging object used to **communicate between different components of an Android application**.

Example uses:

- Start an activity
- Send data between activities

Example

```
val intent = Intent(this, SecondActivity::class.java)

startActivity(intent)
```

---

3. Service

Definition

A **Service** is a component that **runs in the background without a user interface**.

Examples:

- Music playing in background
- File downloading

---

4. Broadcast Receiver

Definition

A **Broadcast Receiver** responds to **system-wide broadcast messages**.

Examples:

- Battery low notification
- Incoming SMS alert

---

5. Content Provider

Definition

A **Content Provider** is used to **share data between different applications**.

Example:

- Sharing contacts between apps

---

6. Android Emulator

Definition

An **Android Emulator** is a virtual device that **simulates an Android device on a computer** for testing applications.

---

7. APK (Android Package)

Definition

An **APK file** is the **installation package used to distribute Android applications**.

Conclusion : Android terminologies such as **Activity, Intent, Service, Broadcast Receiver, Content Provider, Emulator, and APK** are fundamental concepts that help developers understand and build Android applications effectively.

Application Context, Activities, Services, Intents (Android)

These are **important Android components used in application development**. They help manage application behavior, user interaction, background tasks, and communication between components.

---

1. Application Context

Definition

Application Context is an object that provides **global information about the application environment**.
It allows access to application resources, system services, and application-level operations.

In Android, **Context** acts as an interface between the **application and the Android operating system**.

Uses of Application Context

1. Access application resources.
2. Start activities and services.
3. Access system services.
4. Manage application preferences and databases.

Example

val context = applicationContext

Example using context:

Toast.makeText(applicationContext, "Hello Android", Toast.LENGTH_SHORT).show()

Types of Context

1. **Application Context** – Available throughout the entire application lifecycle.
2. **Activity Context** – Available only within an activity.

Key Points

- Provides access to **application-level resources**.
- Used when a **long-lived context is required**.
- Avoid memory leaks by using the correct context.

---

2. Activities

Definition

An **Activity** is a component that represents **a single screen with a user interface in an Android application**.

Activities allow users to **interact with the application**.

Examples of activities:

- Login screen
- Registration screen
- Dashboard screen

Activity Lifecycle

An activity follows a lifecycle consisting of several methods:

Method	Description
`onCreate()`	Called when activity is created
`onStart()`	Activity becomes visible
`onResume()`	Activity starts interacting with user
`onPause()`	Activity partially hidden
`onStop()`	Activity no longer visible
`onDestroy()`	Activity destroyed

Example

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```

Importance of Activities

1. Provides **user interface**.
2. Handles **user interactions**.
3. Manages screen navigation.

---

3. Services

Definition

A **Service** is an Android component that **runs in the background without a user interface**.

Services perform **long-running tasks** even when the user is not interacting with the application.

Examples of Services

- Music playing in background
- Downloading files
- Data synchronization
- Location tracking

Types of Services

1. Started Service

Runs until it finishes its task or is stopped.

2. Bound Service

Allows other components to **bind and communicate with the service**.

Example

```
class MyService : Service() {

    override fun onBind(intent: Intent?): IBinder? {

        return null

    }

}
```

Key Points

- Runs in the **background**.
- Does not provide a user interface.
- Used for **long-running operations**.

---

4. Intents

Definition

An **Intent** is a messaging object used to **request an action from another component**.

It enables communication between components such as:

- Activities
- Services
- Broadcast Receivers

Types of Intents

1. Explicit Intent

Used to start a **specific activity or component**.

Example:

```
val intent = Intent(this, SecondActivity::class.java)

startActivity(intent)
```

---

2. Implicit Intent

Used to request an action from **any application capable of performing that action**.

Example:

```
val intent = Intent(Intent.ACTION_VIEW)

startActivity(intent)
```

Uses of Intents

1. Start activities
2. Start services
3. Send broadcast messages
4. Pass data between components

Example of Passing Data Using Intent

```
val intent = Intent(this, SecondActivity::class.java)

intent.putExtra("name", "Harshal")

startActivity(intent)
```

Conclusion: In Android development, **Application Context provides access to application resources**, **Activities manage user interfaces**, **Services perform background tasks**, and **Intents enable communication between application components**. These components are fundamental for building Android applications.

Receiving and Broadcasting Intents (Android)

Definition

Broadcasting Intents is a mechanism in Android used to **send system-wide messages**, while **Receiving Intents** refers to the process of **listening for and responding to those broadcast messages**.

Android uses a component called **Broadcast Receiver** to handle broadcasted intents.

Broadcast messages can originate from:

- The **Android system**
- **Other applications**
- **The same application**

Broadcasting Intents

Definition

Broadcasting an Intent means sending a message that **notifies multiple applications or components that a specific event has occurred**.

These messages are sent using the **sendBroadcast() method**.

Syntax

sendBroadcast(intent)

Example

```
val intent = Intent("com.example.CUSTOM_INTENT")

sendBroadcast(intent)
```

Explanation

- A broadcast message with the action `"com.example.CUSTOM_INTENT"` is sent.
- Any receiver registered for this action will receive the broadcast.

Receiving Intents

Definition

Receiving Intents means capturing broadcast messages using a **Broadcast Receiver** and performing an action when the broadcast is received.

Broadcast Receivers respond to both:

- **System broadcasts**
- **Application broadcasts**

Broadcast Receiver

Definition

A **Broadcast Receiver** is an Android component that **listens for broadcast intents and responds when they occur**.

Syntax

```
class MyReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {

        // Code executed when broadcast is received

    }

}
```

Example

```
class MyReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {

        Toast.makeText(context, "Broadcast Received", Toast.LENGTH_SHORT).show()

    }

}
```

Registering a Broadcast Receiver

Broadcast receivers must be registered in the **AndroidManifest.xml** file.

Example

```
<receiver android:name=".MyReceiver">
    <intent-filter>
        <action android:name="com.example.CUSTOM_INTENT"/>
    </intent-filter>
</receiver>
```

Types of Broadcasts

1. Normal Broadcast

- Sent using `sendBroadcast()`
- Delivered to all registered receivers

2. Ordered Broadcast

- Sent using `sendOrderedBroadcast()`
- Delivered to receivers **one at a time in priority order**

Examples of System Broadcast Events

Android system sends broadcasts for events such as:

- Battery low
- Device boot completed
- Network connectivity changes
- Incoming SMS

Example action:

android.intent.action.BOOT_COMPLETED

Advantages of Broadcast Intents

1. Enables **communication between applications**.
2. Allows apps to respond to **system events**.
3. Supports **event-driven programming**.
4. Improves **application flexibility and responsiveness**.

Conclusion: Receiving and broadcasting intents in Android allow applications and system components to communicate through broadcast messages. Broadcast intents are sent using the **sendBroadcast() method**, and they are received by **Broadcast Receivers**, which respond to specific events or actions.

Android Manifest File and Its Common Settings

Definition

The **Android Manifest File** (`AndroidManifest.xml`) is the **main configuration file of an Android application**.
It provides essential information about the application to the Android operating system.

This file tells the Android system **how the application should run and interact with other components**.

Every Android project must contain an **AndroidManifest.xml file** located in the **root directory of the project**.

Structure of Android Manifest File

Basic structure:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">
    <application
        android:label="My Application"
        android:icon="@mipmap/ic_launcher">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Main Elements of Android Manifest File

1. `<manifest>` Element

Definition

The `<manifest>` tag is the **root element of the AndroidManifest.xml file**.
It contains information about the **package name and application components**.

Example

```
<manifest package="com.example.myapp">
```

Purpose

- Defines the **package name** of the application.
- Identifies the application uniquely in the Android system.

---

2. `<application>` Element

Definition

The `<application>` element defines **global settings of the application**.

It contains declarations of:

- Activities
- Services
- Broadcast receivers
- Content providers

Example

```
<application
    android:label="MyApp"
    android:icon="@mipmap/ic_launcher">
```

Common Attributes

Attribute	Description
`android:label`	Application name
`android:icon`	Application icon
`android:theme`	Application theme

---

3. `<activity>` Element

Definition

The `<activity>` element declares an **activity (screen) in the application**.

Each screen in an Android app is represented by an activity.

Example

```
<activity android:name=".MainActivity">

</activity>
```

Purpose

- Registers activities with the Android system.
- Specifies which activity will run.

---

4. `<intent-filter>` Element

Definition

The **intent filter** specifies the **types of intents an activity can respond to**.

Example

```
<intent-filter>
    <action android:name="android.intent.action.MAIN"/>
    <category android:name="android.intent.category.LAUNCHER"/>
</intent-filter>
```

Explanation

Element	Purpose
`action MAIN`	Specifies the main entry point
`category LAUNCHER`	Makes the activity appear in the app launcher

5. `<uses-permission>` Element

Definition

The `<uses-permission>` element declares the **permissions required by the application** to access system resources.

Example

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Examples of Permissions

Permission	Purpose
INTERNET	Access internet
CAMERA	Use camera
ACCESS_FINE_LOCATION	Access GPS location

---

6. `<service>` Element

Definition

The `<service>` element declares a **service component** used for background processing.

Example

```
<service android:name=".MyService"/>
```

---

7. `<receiver>` Element

Definition

The `<receiver>` element registers a **Broadcast Receiver** that listens to broadcast messages.

Example

```
<receiver android:name=".MyReceiver"/>
```

Common Settings in Android Manifest

Important settings commonly defined in the manifest file:

Setting	Description
Package Name	Unique identifier of the app
Application Name	Name displayed to users
Application Icon	Icon shown in launcher
Activities	Screens of the app
Permissions	Access to system resources
Services	Background tasks
Broadcast Receivers	Handling broadcast messages

Importance of Android Manifest File

1. Defines the **structure of the Android application**.
2. Registers **all application components**.
3. Declares **permissions required by the application**.
4. Specifies the **entry point of the application**.
5. Provides configuration information to the Android system.

Conclusion: The **AndroidManifest.xml** file is an essential configuration file in Android applications. It defines the application structure, registers components such as activities and services, specifies permissions, and provides important information required for the Android system to run the application properly.

Using Intent Filter and Permissions in Android

1. Intent Filter

Definition

An **Intent Filter** is a component in Android that **specifies the types of intents that a component (such as an activity, service, or broadcast receiver) can respond to**.

It allows the Android system to **determine which application component should handle a particular intent**.

Intent filters are defined in the **AndroidManifest.xml file**.

Purpose of Intent Filter

1. Allows components to **respond to implicit intents**.
2. Enables **communication between applications**.
3. Specifies the **actions an application can perform**.
4. Helps the Android system decide **which app should handle the request**.

Syntax

```
<intent-filter>
    <action android:name="action_name"/>
    <category android:name="category_name"/>
</intent-filter>
```

Elements of Intent Filter

1. Action

The **action** defines the **operation to be performed**.

Example:

```
<action android:name="android.intent.action.VIEW"/>
```

---

2. Category

The **category** provides additional information about the type of component that should handle the intent.

Example:

```
<category android:name="android.intent.category.DEFAULT"/>
```

---

3. Data (Optional)

The **data element** specifies the **type of data handled by the intent**.

Example:

```
<data android:mimeType="text/plain"/>
```

Example of Intent Filter in AndroidManifest.xml

```
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
    </intent-filter>
</activity>
```

Explanation

- `MAIN` specifies the **main entry point of the application**.
- `LAUNCHER` makes the application **appear in the device app launcher**.

Uses of Intent Filters

1. Launching an application.
2. Handling specific types of data.
3. Allowing other applications to interact with your app.

---

2. Permissions in Android

Definition

Permissions are security mechanisms in Android that **control access to sensitive system resources and user data**.

Applications must request permissions before accessing resources such as:

- Internet
- Camera
- Contacts
- Location

Permissions are declared in the **AndroidManifest.xml file**.

Syntax

<uses-permission android:name="permission_name"/>

Example

<uses-permission android:name="android.permission.INTERNET"/>

This permission allows the application to **access the internet**.

Common Android Permissions

Permission	Description
INTERNET	Access internet
CAMERA	Use device camera
ACCESS_FINE_LOCATION	Access GPS location
READ_CONTACTS	Read user contacts
WRITE_EXTERNAL_STORAGE	Write data to storage

Types of Permissions

1. Normal Permissions

Permissions that **do not pose a risk to user privacy**.

Examples:

- Access network state
- Set wallpaper

---

2. Dangerous Permissions

Permissions that **may affect user privacy or device security**.

Examples:

- Camera access
- Location access
- Contacts access

These permissions require **user approval at runtime**.

Importance of Permissions

1. Protects **user data and privacy**.
2. Ensures **secure application behavior**.
3. Prevents **unauthorized access to system resources**.
4. Allows Android to **control application capabilities**.

Conclusion: Intent Filters allow Android components to respond to specific intents and enable communication between applications. Permissions provide a security mechanism that controls access to sensitive system resources, ensuring safe and secure Android application operation.

Managing Application Resources in a Hierarchy (Android)

Definition

In Android, **application resources** are external files used by the application such as **layouts, images, strings, colors, and styles**. These resources are organized in a **hierarchical structure inside the `res` (resources) directory**.

Managing resources in a hierarchy helps Android **efficiently organize, access, and maintain application resources**.

Resource Directory Structure

All application resources are stored inside the **res folder**.

Example hierarchy:

res/

  layout/

  drawable/

  mipmap/

  values/

  menu/

  raw/

  anim/

Each folder contains specific types of resources used in the application.

Important Resource Directories

1. layout

Definition

The **layout folder** contains **XML files used to design the user interface of the application**.

Example file:

activity_main.xml

Example code:

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello Android"/>
```

Purpose:

- Defines UI elements
- Controls screen layout

---

2. drawable

Definition

The **drawable folder** contains **image resources** used in the application.

Examples:

- PNG images
- JPG images
- Vector graphics

Example:

logo.png

background.jpg

Usage example:

```
android:src="@drawable/logo"
```

---

3. mipmap

Definition

The **mipmap folder** stores **launcher icons for the application**.

Example:

ic_launcher.png

Usage:

```
android:icon="@mipmap/ic_launcher"
```

---

4. values

Definition

The **values folder** stores simple resource values in XML format.

Common files include:

File	Purpose
strings.xml	Store text strings
colors.xml	Store color values
styles.xml	Define UI styles
dimens.xml	Store dimensions

Example (strings.xml):

```
<resources>
    <string name="app_name">MyApplication</string>
</resources>
```

Usage in code:

```
@string/app_name
```

---

5. menu

Definition

The **menu folder** contains XML files used to define **application menus**.

Example:

main_menu.xml

Example code:

```
<menu>
    <item android:title="Settings"/>
</menu>
```

---

6. raw

Definition

The **raw folder** stores **raw files such as audio, video, or text files**.

Example:

music.mp3

data.json

---

7. anim

Definition

The **anim folder** contains **animation files used for UI animations**.

Example:

fade_in.xml

slide_out.xml

Accessing Resources in Android

Resources are accessed using the **R class**.

Example in Kotlin:

```
setContentView(R.layout.activity_main)
```

Example for string:

```
textView.text = getString(R.string.app_name)
```

Advantages of Resource Hierarchy

1. Organizes application resources systematically.
2. Makes resource management easier.
3. Allows reuse of resources.
4. Supports multiple screen sizes and languages.
5. Improves application maintainability.

Conclusion: In Android, application resources are organized in a hierarchical structure inside the **res directory**, which includes folders such as **layout, drawable, mipmap, values, menu, raw, and anim**. This hierarchical organization helps developers efficiently manage and access resources while building Android applications.

Working with Different Types of Resources in Android

Definition

In Android, **resources** are external files used in an application such as **layouts, images, strings, colors, menus, and animations**.
These resources are stored in the **res (resources) directory** and help developers separate the **application code from the user interface and other resources**.

Working with different types of resources allows developers to **manage application components efficiently and improve maintainability**.

Types of Resources in Android

1. Layout Resources

Definition

Layout resources define the **structure and user interface (UI) of an Android application**.
They are written in **XML files** and stored in the **res/layout folder**.

Example File : Activity_main.xml

Example Code

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello Android"/>
```

Usage in Kotlin : setContentView(R.layout.activity_main)

---

2. String Resources

Definition

String resources store text values used in the application.
They are stored in the **res/values/strings.xml file**.

This helps in **easy text management and language localization**.

Example

```
<resources>
    <string name="app_name">My Application</string>
</resources>
```

Usage

```
android:text="@string/app_name"
```

---

3. Drawable Resources

Definition

Drawable resources are images used in the application such as **icons, backgrounds, and graphics**.

They are stored in the **res/drawable folder**.

Examples

- PNG images
- JPG images
- Vector graphics

Example usage:

```
android:src="@drawable/logo"
```

---

4. Color Resources

Definition

Color resources store color values used in the application UI.

They are stored in the **res/values/colors.xml file**.

Example

```
<resources>
    <color name="primaryColor">#FF0000</color>
</resources>
```

Usage : android:textColor="@color/primaryColor"

---

5. Menu Resources

Definition

Menu resources define menus used in an application such as **options menus or context menus**.

They are stored in the **res/menu folder**.

Example

```
<menu>
   <item android:title="Settings"/>
</menu>
```

---

6. Animation Resources

Definition

Animation resources define animations used in Android applications.

They are stored in the **res/anim folder**.

Example:

fade_in.xml

slide_out.xml

Example animation code:

```
<alpha
   android:fromAlpha="0.0"
   android:toAlpha="1.0"
   android:duration="1000"/>
```

---

7. Raw Resources

Definition

The **raw resource folder** stores raw files such as **audio, video, and text files**.

Location:

res/raw

Example files:

music.mp3

video.mp4

data.txt

Accessing Resources in Android

Android uses the **R class** to access resources.

Example:

R.layout.activity_main

R.string.app_name

R.drawable.logo

Advantages of Using Resources

1. Separates **UI design from program logic**.
2. Makes applications **easy to maintain and update**.
3. Supports **multiple languages and screen sizes**.
4. Improves **code readability and organization**.

Conclusion: Android applications use various types of resources such as **layout, strings, drawables, colors, menus, animations, and raw files**. These resources are stored in the **res directory** and accessed through the **R class**, allowing developers to efficiently manage and organize application components.

Unit–3: Android User Interface Design

User Interface Screen Elements

Definition : In Android, **User Interface (UI) elements** are graphical components that allow users to **interact with an application**.
These elements are also called **widgets** and are used to create the **layout and functionality of the application screen**.

UI elements are generally defined in **XML layout files** located in the **res/layout directory**.

---

1. Button

Definition : A **Button** is a UI component that allows the user to **perform an action when clicked**.

Example XML

```
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit"/>
```

Example Kotlin Code

```
val button = findViewById<Button>(R.id.btnSubmit)
button.setOnClickListener {
    println("Button Clicked")
}
```

Uses

- Submit forms
- Start activities
- Trigger events

2. EditText

Definition : EditText is a UI component used to **receive text input from the user**.

Example

```
<EditText
    android:id="@+id/editName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter Name"/>
```

Uses

- Login forms
- User input fields
- Search boxes

---

3. TextView

Definition : TextView is used to **display text on the screen**.

Example

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Welcome to Android"/>
```

Uses

- Display messages
- Show labels
- Display data

---

4. DatePicker

Definition : DatePicker is a UI component that allows the user to **select a date**.

Example

```
<DatePicker
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>
```

Uses

- Selecting birth date
- Booking dates
- Scheduling events

---

5. TimePicker

Definition : TimePicker allows users to **select time (hours and minutes)**.

Example

```
<TimePicker

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"/>
```

Uses

- Alarm setting
- Meeting scheduling
- Time selection

---

6. ProgressBar

Definition : A **ProgressBar** shows the **progress of an operation** such as downloading or loading data.

Example

```
<ProgressBar

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"/>
```

Uses

- File downloading
- Loading indicators
- Background processing

---

7. ListView

Definition : A **ListView** displays a **scrollable list of items vertically**.

Example

```
<ListView

    android:id="@+id/listView"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"/>
```

Uses

- Display lists of data
- Contact lists
- Menu options

---

8. GridView

Definition : A **GridView** displays items in a **two-dimensional grid format**.

Example

```
<GridView

    android:id="@+id/gridView"

    android:layout_width="match_parent"

    android:layout_height="wrap_content"

    android:numColumns="2"/>
```

Uses

- Image galleries
- Product displays
- Photo grids

---

9. RadioGroup

Definition : A **RadioGroup** is used to group multiple **RadioButtons**, allowing the user to **select only one option at a time**.

Example

```
<RadioGroup

   android:layout_width="wrap_content"

   android:layout_height="wrap_content">

   <RadioButton

      android:text="Male"/>

   <RadioButton

      android:text="Female"/>

</RadioGroup>
```

Uses

- Gender selection
- Multiple choice options

---

10. ImageButton

Definition : An **ImageButton** is a button that **displays an image instead of text**.

Example

```
<ImageButton

   android:layout_width="wrap_content"

   android:layout_height="wrap_content"

   android:src="@drawable/icon"/>
```

Uses

- Toolbar icons
- Camera button
- Media controls

---

11. Fragment

Definition

A **Fragment** is a reusable part of an activity that contains its own **UI and functionality**.

Fragments allow developers to **create flexible and modular user interfaces**.

Example Kotlin Code

```
class ExampleFragment : Fragment() {
```

```
    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        return inflater.inflate(R.layout.fragment_example, container, false)
    }
}
```

Advantages of Fragments

1. Reusable UI components
2. Better screen adaptability
3. Improved UI modularity
4. Supports tablets and large screens

Conclusion : Android UI screen elements such as **Button, EditText, TextView, DatePicker, TimePicker, ProgressBar, ListView, GridView, RadioGroup, ImageButton, and Fragment** are essential components used to design interactive and functional user interfaces in Android applications.

Designing User Interfaces with Layouts in Android

Definition : In Android, **Layouts** are used to **arrange and organize UI components (widgets) on the screen**.
Layouts define the **structure of the user interface** by positioning elements such as buttons, text fields, and images.

Layouts are usually defined in **XML files inside the `res/layout` directory**.

Android provides several types of layouts such as:

- **LinearLayout**
- **RelativeLayout**
- **TableLayout**
- FrameLayout
- ConstraintLayout (modern layout)

---

1. Linear Layout

Definition : **LinearLayout** is a layout that arranges UI elements **in a single row or column**.

It can organize components:

- **Vertically**
- **Horizontally**

This layout is commonly used for **simple UI designs**.

Syntax

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
</LinearLayout>
```

Example

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:text="Username"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

Advantages

1. Simple and easy to use.
2. Good for **simple UI designs**.
3. Supports **horizontal and vertical arrangements**.

---

2. Relative Layout

Definition

RelativeLayout arranges UI components **relative to other elements or the parent layout**.

This allows more **flexible UI design compared to LinearLayout**.

Syntax

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
</RelativeLayout>
```

Example

```
<RelativeLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:id="@+id/text1"
        android:text="Login"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
```

```
    android:text="Submit"
    android:layout_below="@id/text1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"/>
</RelativeLayout>
```

Common Attributes

Attribute	Description
layout_below	Place element below another
layout_above	Place element above another
layout_toRightOf	Place element to the right
layout_toLeftOf	Place element to the left

Advantages

1. Flexible UI arrangement.
2. Reduces nested layouts.
3. Suitable for **complex UI designs**.

---

3. Table Layout

Definition

TableLayout arranges UI elements in **rows and columns similar to a table**.

Each row is defined using **TableRow**.

Syntax

```
<TableLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">
</TableLayout>
```

Example

```
<TableLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TableRow>
```

```
    <TextView android:text="Name"/>
    <EditText/>
  </TableRow>
  <TableRow>
    <TextView android:text="Password"/>
    <EditText/>
  </TableRow>
</TableLayout>
```

Advantages

1. Organizes UI elements in **tabular format**.
2. Useful for **forms and data entry screens**.
3. Easy to align fields in rows and columns.

Comparison of Layouts

Layout	Arrangement	Best Use
LinearLayout	Horizontal or Vertical	Simple UI
RelativeLayout	Relative positioning	Complex UI
TableLayout	Rows and columns	Forms and structured data

Importance of Layouts in Android

1. Defines **UI structure of the application**.
2. Organizes UI components properly.
3. Improves **user experience**.
4. Supports different screen sizes.

Conclusion : Layouts are essential components in Android UI design. **LinearLayout, RelativeLayout, and TableLayout** help developers organize interface elements efficiently and create structured, user-friendly mobile application screens.

Dialogs in Android

Definition

A **Dialog** in Android is a **small window that appears in front of the activity to display important information or request input from the user**.
It temporarily interrupts the current activity and requires the user to take an action before continuing.

Dialogs are commonly used for:

- Confirmations
- Alerts
- User input
- Selecting options

Types of Dialogs in Android

Android provides several types of dialogs:

1. **AlertDialog**
2. **ProgressDialog**
3. **DatePickerDialog**
4. **TimePickerDialog**
5. **Custom Dialog**

---

1. AlertDialog

Definition

An **AlertDialog** is the most commonly used dialog in Android.
It displays a message with **buttons such as OK, Cancel, Yes, or No**.

Syntax (Kotlin)

```
val builder = AlertDialog.Builder(this)

builder.setTitle("Alert")

builder.setMessage("Do you want to exit?")

builder.setPositiveButton("Yes") { dialog, which ->

    finish()

}

builder.setNegativeButton("No") { dialog, which ->

    dialog.dismiss()

}

builder.show()
```

Uses

- Confirming actions
- Warning messages
- User decisions

---

2. ProgressDialog

Definition : A **ProgressDialog** is used to show the **progress of a task such as downloading or loading data**.

Example

```
val progressDialog = ProgressDialog(this)

progressDialog.setMessage("Loading...")

progressDialog.show()
```

Uses

- File downloading
- Data processing
- Background operations

3. DatePickerDialog

Definition : A **DatePickerDialog** allows users to **select a date from a calendar interface**.

Example

```
val datePicker = DatePickerDialog(
    this,
    { _, year, month, day ->
        println("$day/$month/$year")
    },
    2024,
    1,
    1
)
datePicker.show()
```

Uses

- Selecting birth date
- Booking events
- Scheduling tasks

---

4. TimePickerDialog

Definition : A **TimePickerDialog** allows users to **select time (hours and minutes)**.

Example

```
val timePicker = TimePickerDialog(
    this,
    { _, hour, minute ->
        println("$hour:$minute")
    },
    12,
    0,
    true
)
timePicker.show()
```

Uses

- Setting alarms
- Scheduling meetings
- Selecting appointment times

---

5. Custom Dialog

Definition

A **Custom Dialog** allows developers to **design their own dialog layout using XML**.

This provides more flexibility compared to standard dialogs.

Example

```
val dialog = Dialog(this)

dialog.setContentView(R.layout.custom_dialog)

dialog.show()
```

Advantages of Dialogs

1. Displays important messages to the user.
2. Helps in user confirmation and decision making.
3. Improves user interaction.
4. Allows quick user input.

Conclusion : Dialogs in Android are used to interact with users by displaying messages, requesting input, or showing progress. Types of dialogs include **AlertDialog, ProgressDialog, DatePickerDialog, TimePickerDialog, and Custom Dialog**, each serving different purposes in Android applications.

Drawing and Working with Animation in Android

Definition

Animation in Android is a technique used to create the **illusion of movement by displaying a sequence of images or frames**. Animations improve the **visual appearance and user experience of Android applications**.

Android supports several types of animations such as:

- Frame by Frame Animation
- Tween Animation
- Property Animation

Frame By Frame Animation

Definition

Frame by Frame Animation is a type of animation where **a sequence of drawable images is displayed one after another** to create a motion effect.

Each image represents a **frame**, and when frames are displayed quickly in sequence, they create the appearance of movement.

This animation is also called **Drawable Animation**.

Working Principle

Frame by frame animation works by:

1. Creating multiple images representing different stages of motion.
2. Defining these images in an **animation list XML file**.
3. Displaying them sequentially on an **ImageView**.

Steps to Create Frame By Frame Animation

Step 1: Create Image Frames

Prepare multiple images such as:

frame1.png

frame2.png

frame3.png

frame4.png

Store them in the **res/drawable folder**.

Step 2: Create Animation XML File

Create a file inside **res/drawable** such as:

animation.xml

Example

```
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false">
    <item android:drawable="@drawable/frame1" android:duration="200"/>
    <item android:drawable="@drawable/frame2" android:duration="200"/>
    <item android:drawable="@drawable/frame3" android:duration="200"/>
    <item android:drawable="@drawable/frame4" android:duration="200"/>
</animation-list>
```

Explanation

Attribute	Meaning
animation-list	Defines frame animation
drawable	Image frame
duration	Time each frame is displayed
oneshot	false means animation repeats

Step 3: Use Animation in Layout

Example XML layout:

```
<ImageView
    android:id="@+id/imageView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/animation"/>
```

Step 4: Start Animation Using Kotlin

Example code:

```
val imageView = findViewById<ImageView>(R.id.imageView)

val animation = imageView.drawable as AnimationDrawable

animation.start()
```

Example Applications of Frame Animation

- Loading animations
- Game character movements
- Animated icons
- Cartoon or GIF-like animations

Advantages of Frame Animation

1. Easy to implement.
2. Creates smooth visual effects.
3. Useful for simple animations.

Disadvantages

1. Requires multiple image files.
2. Consumes more memory.
3. Not suitable for complex animations.

Conclusion : Frame by Frame Animation in Android displays a sequence of images to create motion. It is implemented using **animation-list XML files and drawable resources**, and it is commonly used for simple visual effects and animated graphics in Android applications.

Tweened Animation in Android

Definition

Tweened Animation (also called **Tween Animation**) is a type of animation in Android where **intermediate frames are automatically calculated between the starting and ending positions of an object**.

Instead of defining each frame manually, Android calculates the transformation such as **movement, rotation, scaling, or fading** between two states.

Tweened animation is mainly used to create **simple visual effects for UI components**.

Types of Tweened Animations

Android provides four main types of tween animations:

1. **Translate Animation**
2. **Scale Animation**
3. **Rotate Animation**
4. **Alpha Animation**

---

1. Translate Animation

Definition

Translate Animation moves a UI element from one position to another on the screen.

Example XML

```
<translate
    android:fromXDelta="0%"
    android:toXDelta="100%"
    android:duration="1000"/>
```

Explanation

Attribute	Description
fromXDelta	Starting horizontal position
toXDelta	Ending horizontal position
duration	Time of animation

---

2. Scale Animation

Definition

Scale Animation changes the **size of an object**.

Example XML

```
<scale
    android:fromXScale="1.0"
    android:toXScale="2.0"
    android:duration="1000"/>
```

Explanation

The object becomes **twice its original size**.

---

3. Rotate Animation

Definition

Rotate Animation rotates an object around a specific point.

Example XML

```
<rotate
    android:fromDegrees="0"
    android:toDegrees="360"
    android:duration="1000"/>
```

Explanation

The object rotates **360 degrees**.

---

4. Alpha Animation

Definition

Alpha Animation changes the **opacity (transparency) of a view**.

Example XML

```
<alpha
    android:fromAlpha="0.0"
    android:toAlpha="1.0"
    android:duration="1000"/>
```

Explanation

The object gradually becomes **visible**.

Steps to Implement Tween Animation

Step 1: Create Animation File

Create a file inside:

res/anim/

Example:

res/anim/rotate.xml

Step 2: Define Animation

Example:

```
<rotate
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:fromDegrees="0"
    android:toDegrees="360"
    android:duration="1000"/>
```

Step 3: Apply Animation in Kotlin

Example code:

```
val image = findViewById<ImageView>(R.id.imageView)

val animation = AnimationUtils.loadAnimation(this, R.anim.rotate)

image.startAnimation(animation)
```

Advantages of Tween Animation

1. Easy to implement.
2. Requires less memory compared to frame animation.
3. Automatically calculates intermediate frames.
4. Suitable for UI effects.

Disadvantages

1. Limited control over individual frames.
2. Less flexible for complex animations.

Difference Between Frame Animation and Tween Animation

Feature	Frame Animation	Tween Animation
Frames	Defined manually	Generated automatically
Images	Requires multiple images	Uses transformations
Memory usage	High	Low
Complexity	Simple motion graphics	UI transformations

Conclusion : **Tweened Animation** in Android automatically generates intermediate frames to animate UI elements. It supports transformations such as **translation, scaling, rotation, and alpha effects**, making it efficient for creating smooth UI animations in Android applications.

Unit–4: Database Connectivity Using SQLite and Content Provider

Using Android Data and Storage APIs

Definition

Android provides **Data and Storage APIs** that allow applications to **store, manage, and retrieve data on a device**. These APIs help developers store data **locally on the device or remotely** depending on the application requirements.

Data storage is essential for saving information such as **user settings, application data, files, and database records**.

Types of Data Storage in Android

Android provides several data storage options through its APIs:

1. **Shared Preferences**
2. **Internal Storage**
3. **External Storage**
4. **SQLite Databases**
5. **Network Storage**

Each method is used for different types of data.

---

1. Shared Preferences

Definition

Shared Preferences is used to store **small amounts of data in the form of key-value pairs**.

It is commonly used for saving:

- User settings
- Login information
- Application preferences

Example (Kotlin)

```
val sharedPref = getSharedPreferences("MyPrefs", MODE_PRIVATE)

val editor = sharedPref.edit()

editor.putString("username", "Harshal")

editor.apply()
```

Reading Data

```
val username = sharedPref.getString("username", "Default")
```

---

2. Internal Storage

Definition

Internal Storage stores data **privately within the device memory**, and the data is accessible **only by the application that created it**.

Files stored here are **automatically deleted when the application is uninstalled**.

Example

```
val fileOutput = openFileOutput("data.txt", MODE_PRIVATE)

fileOutput.write("Hello Android".toByteArray())

fileOutput.close()
```

---

3. External Storage

Definition

External Storage stores files that can be **shared with other applications and accessed by the user**.

Examples include:

- SD cards
- Public device storage

Example

```
val file = File(getExternalFilesDir(null), "data.txt")

file.writeText("Hello Android")
```

---

4. SQLite Database

Definition

SQLite is a lightweight **relational database management system** used to store structured data inside Android applications.

It allows developers to store data in **tables consisting of rows and columns**.

Example

```
val db = openOrCreateDatabase("StudentDB", MODE_PRIVATE, null)

db.execSQL("CREATE TABLE IF NOT EXISTS Student(id INTEGER, name TEXT)")
```

5. Network Storage

Definition

Network storage allows Android applications to **store and retrieve data from remote servers using APIs and web services**.

Examples include:

- Cloud databases
- Web APIs
- Online storage systems

Advantages of Android Data Storage APIs

1. Provides **multiple options for storing data**.
2. Helps manage both **small and large datasets**.
3. Ensures **secure and efficient data management**.
4. Supports both **local and remote data storage**.

Conclusion : Android Data and Storage APIs provide various mechanisms such as **Shared Preferences, Internal Storage, External Storage, SQLite Databases, and Network Storage** to store and manage application data efficiently. Developers choose the appropriate storage method depending on the **type and size of data required by the application**.

<u>Managing Data Using SQLite in Android</u>

Definition

SQLite is a lightweight **relational database management system (RDBMS)** used in Android applications to **store, manage, and retrieve structured data**.

SQLite stores data in the form of **tables consisting of rows and columns**. It is embedded within Android, so it does not require a separate database server.

SQLite is commonly used to store:

- User information
- Application data
- Offline data storage

Features of SQLite

1. **Lightweight database system**
2. **Embedded in Android**
3. Supports **SQL queries**
4. Stores data in **tables**
5. Does not require a separate server

SQLite Database Components

SQLite database mainly includes:

1. **Database**
2. **Tables**
3. **Columns**
4. **Rows**

Steps to Manage Data Using SQLite

Step 1: Create Database

SQLite databases can be created using the **SQLiteOpenHelper class**.

Example

```
class DBHelper(context: Context) : SQLiteOpenHelper(
    context,
    "StudentDB",
    null,
    1
) {
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL("CREATE TABLE Student(id INTEGER PRIMARY KEY, name TEXT)")
    }
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS Student")
        onCreate(db)
    }
}
```

Explanation

Method	Purpose
onCreate()	Creates database tables
onUpgrade()	Updates database structure

Step 2: Insert Data

Data can be inserted using **SQL INSERT queries**.

Example

```
val db = writableDatabase
db.execSQL("INSERT INTO Student VALUES(1,'Harshal')")
```

Step 3: Retrieve Data

Data can be retrieved using **SELECT queries**.

Example

```
val cursor = db.rawQuery("SELECT * FROM Student", null)
while(cursor.moveToNext()) {
    val id = cursor.getInt(0)
    val name = cursor.getString(1)
}
```

Step 4: Update Data

Data can be updated using **UPDATE queries**.

Example

db.execSQL("UPDATE Student SET name='Raj' WHERE id=1")

Step 5: Delete Data

Data can be deleted using **DELETE queries**.

Example

db.execSQL("DELETE FROM Student WHERE id=1")

CRUD Operations in SQLite

SQLite supports four basic database operations:

Operation	SQL Command
Create	CREATE TABLE
Read	SELECT
Update	UPDATE
Delete	DELETE

These operations are known as **CRUD operations**.

Advantages of SQLite

1. Lightweight and fast database system.
2. No server installation required.
3. Supports standard SQL queries.
4. Ideal for **mobile application data storage**.

Conclusion : SQLite is a built-in relational database in Android used for storing structured data. Developers can manage data efficiently using SQL commands such as **CREATE, INSERT, SELECT, UPDATE, and DELETE**, making SQLite suitable for handling application data in Android applications.

Sharing Data Between Applications with Content Providers (Android)

Definition

A **Content Provider** is an Android component that allows **data to be shared securely between different applications**.
It provides a **standard interface to access and manage application data** such as databases, files, or other storage systems.

Content providers use **URIs (Uniform Resource Identifiers)** to identify data and allow other applications to **read, insert, update, or delete data**.

Purpose of Content Providers

1. To **share application data with other applications**.
2. To **control access to application data securely**.
3. To provide a **standard interface for data management**.
4. To enable **data exchange between different Android applications**.

Examples of Built-in Content Providers

Android already provides several built-in content providers such as:

Content Provider	Purpose
Contacts Provider	Access user contacts
Media Provider	Access images, audio, and videos
Calendar Provider	Access calendar events

Example: An application can access phone contacts through the **Contacts Content Provider**.

Content Provider Components

A content provider consists of the following elements:

1. **Content Provider Class**
2. **Content URI**
3. **Content Resolver**
4. **Database or Storage**

---

1. Content URI

Definition : A **Content URI** uniquely identifies data in a content provider.

Example

content://com.example.provider/student

Structure

Part	Description
content://	Protocol used for content providers
authority	Name of the content provider
path	Data table or resource

---

2. Content Resolver

Definition : A **Content Resolver** is used by applications to **communicate with content providers**.

It performs operations such as:

- Query
- Insert
- Update
- Delete

Example

```
val cursor = contentResolver.query(uri, null, null, null, null)
```

---

3. Creating a Content Provider

A content provider class must extend the **ContentProvider** class.

Example

```
class MyProvider : ContentProvider() {

    override fun onCreate(): Boolean {

        return true

    }

    override fun query(uri: Uri, projection: Array<String>?, selection: String?, selectionArgs: Array<String>?, sortOrder: String?): Cursor? {

        return null

    }

}
```

Important Methods of Content Provider

Method	Description
onCreate()	Initializes the provider
query()	Retrieves data
insert()	Adds new data
update()	Updates existing data
delete()	Removes data
getType()	Returns MIME type of data

Registering Content Provider in Manifest

A content provider must be declared in the **AndroidManifest.xml** file.

Example

```
<provider

   android:name=".MyProvider"

   android:authorities="com.example.provider"/>
```

Advantages of Content Providers

1. Enables **secure data sharing between applications**.
2. Provides **standardized data access mechanism**.
3. Supports **CRUD operations**.
4. Ensures **controlled access to application data**.

Conclusion : Content Providers in Android allow applications to **share and manage data securely across different applications**. They use **URIs and Content Resolvers** to perform operations such as **query, insert, update, and delete**, making them an essential mechanism for data sharing in Android systems.

Unit :- 5 "Location Based Services (LBS), Common Android API, Notifications, Services, Deployment of applications"

Using Global Positioning Services (GPS) in Android

Definition

Global Positioning System (GPS) is a satellite-based navigation system used to **determine the geographic location of a device on Earth**. In Android applications, GPS is used to obtain the **latitude, longitude, altitude, and speed of a device**.

GPS is commonly used in **Location Based Services (LBS)** such as navigation, maps, and tracking applications.

Features of GPS in Android

1. Provides **accurate location information**.
2. Uses **satellites to determine device position**.
3. Works with Android **Location APIs**.
4. Used for **navigation and tracking applications**.

Components Used for GPS in Android

To use GPS in Android, the following components are required:

1. **Location Manager**
2. **Location Provider**
3. **Location Listener**

---

1. Location Manager

Definition

The **LocationManager** class is used to **access the device's location services**.

It allows applications to **request location updates** from GPS or network providers.

Example

```
val locationManager = getSystemService(Context.LOCATION_SERVICE) as LocationManager
```

---

2. Location Provider

Definition :- A **Location Provider** determines how location data is obtained.

Types of providers include:

Provider	Description
GPS Provider	Uses satellites for accurate location
Network Provider	Uses Wi-Fi and mobile networks
Passive Provider	Uses location updates from other apps

---

3. Location Listener

Definition

A **LocationListener** receives updates when the device location changes.

Example

```
val locationListener = object : LocationListener {
    override fun onLocationChanged(location: Location) {
        val latitude = location.latitude
        val longitude = location.longitude
    }
}
```

Steps to Use GPS in Android

Step 1: Add Permission in Manifest

Android requires permission to access device location.

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

Step 2: Access Location Manager

```
val locationManager = getSystemService(Context.LOCATION_SERVICE) as LocationManager
```

Step 3: Request Location Updates

```
locationManager.requestLocationUpdates(
    LocationManager.GPS_PROVIDER,
    5000,
    5f,
    locationListener
)
```

Example:- Getting Device Location

```
val locationManager = getSystemService(Context.LOCATION_SERVICE) as LocationManager

val locationListener = object : LocationListener {

    override fun onLocationChanged(location: Location) {

        val lat = location.latitude

        val lon = location.longitude

    }

}

locationManager.requestLocationUpdates(

    LocationManager.GPS_PROVIDER,

    5000,

    5f,

    locationListener

)
```

Applications of GPS in Android

1. **Navigation systems**
2. **Google Maps**
3. **Ride-sharing apps**
4. **Location tracking apps**
5. **Delivery services**

Advantages of GPS

1. Provides **accurate location information**.
2. Useful for **navigation and tracking**.
3. Supports **real-time location updates**.
4. Works worldwide with satellite signals.

Limitations

1. Requires **clear satellite signals**.
2. Consumes **battery power**.
3. May not work well **indoors or underground**.

Conclusion : GPS in Android is used to obtain the **device's geographic location using satellites**. By using components such as **LocationManager, LocationProvider, and LocationListener**, Android applications can access and track location data for various Location Based Services (LBS).

Geocoding Locations in Android

Definition

Geocoding is the process of **converting a human-readable address or place name into geographic coordinates such as latitude and longitude**.

Similarly, **Reverse Geocoding** is the process of **converting geographic coordinates (latitude and longitude) into a readable address**.

In Android, geocoding is performed using the **Geocoder class**, which interacts with location services to obtain address information.

Types of Geocoding

1. Forward Geocoding

Converts an **address or location name into geographic coordinates (latitude and longitude)**.

Example:

"Ahmedabad, Gujarat"

→ Latitude: 23.0225

→ Longitude: 72.5714

---

2. Reverse Geocoding

Converts **latitude and longitude into a human-readable address**.

Example:

Latitude: 23.0225

Longitude: 72.5714

→ Ahmedabad, Gujarat, India

Geocoder Class

Definition

The **Geocoder class** in Android is used to **convert between addresses and geographic coordinates**.

It works with location services to retrieve location information.

Example: Reverse Geocoding

Step 1: Create Geocoder Object

```
val geocoder = Geocoder(this, Locale.getDefault())
```

Step 2: Get Address from Latitude and Longitude

```
val addresses = geocoder.getFromLocation(latitude, longitude, 1)
if(addresses != null && addresses.isNotEmpty()) {
    val address = addresses[0].getAddressLine(0)
}
```

Explanation

Parameter	Description
latitude	Device latitude
longitude	Device longitude
1	Maximum number of results

Example: Forward Geocoding

Convert Address to Coordinates

```
val geocoder = Geocoder(this, Locale.getDefault())

val addresses = geocoder.getFromLocationName("Ahmedabad", 1)

if(addresses != null && addresses.isNotEmpty()) {

    val latitude = addresses[0].latitude

    val longitude = addresses[0].longitude

}
```

Applications of Geocoding

1. **Google Maps navigation**
2. **Location-based services (LBS)**
3. **Food delivery applications**
4. **Ride-sharing applications**
5. **Location search systems**

Advantages of Geocoding

1. Converts addresses to coordinates easily.
2. Supports both **forward and reverse geocoding**.
3. Useful for **location-based applications**.
4. Improves navigation and mapping systems.

Limitations

1. Requires **internet connection**.
2. Accuracy may vary depending on location data.
3. May take time to retrieve results.

Conclusion : Geocoding in Android is the process of converting addresses into geographic coordinates and vice versa using the **Geocoder class**. It plays an important role in **Location Based Services (LBS)** such as navigation, maps, and location tracking applications.

Mapping Locations in Android

Definition

Mapping Locations in Android refers to the process of **displaying geographic locations on a map using coordinates such as latitude and longitude**.
It is commonly implemented using **mapping services such as Google Maps API**.

Mapping helps applications **visualize locations, routes, and geographic information on a map interface**.

Google Maps in Android

To display maps in Android applications, developers commonly use the **Google Maps API**, which allows apps to:

- Display maps
- Show user location
- Add markers on locations
- Draw routes between places

Components Required for Mapping

To implement mapping in Android, the following components are required:

1. **Google Maps API**
2. **Map Fragment**
3. **GoogleMap Object**
4. **Latitude and Longitude Coordinates**

Steps to Implement Mapping in Android

Step 1: Add Google Maps Dependency

Add the Google Maps library in the project.

Example:

```
implementation 'com.google.android.gms:play-services-maps'
```

Step 2: Add Google Maps API Key

A **Google Maps API key** is required to use map services.

Example in **AndroidManifest.xml**:

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="YOUR_API_KEY"/>
```

Step 3: Add Map Fragment in Layout

Example XML layout:

```
<fragment
    android:id="@+id/map"
    android:name="com.google.android.gms.maps.SupportMapFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
```

Step 4: Display Map in Activity

Example Kotlin code:

```
class MapsActivity : AppCompatActivity(), OnMapReadyCallback {
    private lateinit var mMap: GoogleMap
    override fun onMapReady(googleMap: GoogleMap) {
        mMap = googleMap
        val location = LatLng(23.0225, 72.5714)
        mMap.addMarker(MarkerOptions().position(location).title("Ahmedabad"))
        mMap.moveCamera(CameraUpdateFactory.newLatLng(location))
    }
}
```

Explanation

Component	Description
LatLng	Represents geographic coordinates
Marker	Displays a location marker on map
moveCamera()	Moves map view to a location

Example Output

The application displays:

- A **map interface**
- A **marker showing a specific location**

Example:

Ahmedabad location marked on Google Map

Applications of Mapping

1. Navigation systems
2. Ride-sharing applications
3. Delivery services
4. Travel and tourism apps
5. Location tracking applications

Advantages of Mapping

1. Provides **visual representation of locations**.
2. Helps in **navigation and route planning**.
3. Improves **user experience in location-based applications**.
4. Supports **real-time location tracking**.

Conclusion : Mapping locations in Android allows developers to display geographic locations using tools such as **Google Maps API**. By using coordinates like **latitude and longitude**, applications can show markers, track locations, and provide navigation features for users.

Location Based Services (LBS) – Additional Features

Definition

Location Based Services (LBS) are mobile services that **use the geographic location of a device to provide useful information or services to users**.
These services combine technologies such as **GPS, mobile networks, maps, and internet connectivity** to determine the location of a user.

LBS are widely used in **navigation, tracking, mapping, and location-aware applications**.

Technologies Used in Location Based Services

Several technologies work together to provide LBS:

1. **GPS (Global Positioning System)** – Determines device location using satellites.
2. **Wi-Fi Positioning** – Determines location using nearby Wi-Fi networks.
3. **Cell Tower Triangulation** – Uses mobile network towers to estimate location.
4. **Geocoding** – Converts addresses to coordinates and vice versa.
5. **Mapping Services** – Displays location visually on maps.

Common Location Based Services

1. Location Tracking

Definition

Location tracking allows applications to **monitor and update the real-time position of a device or user**.

Examples

- Vehicle tracking systems
- Fitness tracking apps
- Child safety tracking apps

---

2. Navigation Services

Definition

Navigation services help users **find routes and directions from one location to another**.

Examples

- Turn-by-turn directions
- Shortest route calculation
- Traffic information

Example applications:

- Navigation apps
- Delivery route planning

---

3. Location Search

Definition

Location search allows users to **search for nearby places based on their current location**.

Examples

- Restaurants nearby
- Hospitals nearby
- ATMs and banks

---

4. Geofencing

Definition

Geofencing creates a **virtual boundary around a geographic area**.
When a device enters or leaves this area, the application triggers an action.

Examples

- Notifications when entering a store
- Security alerts
- Smart home automation

---

5. Emergency Location Services

Definition

Emergency location services help authorities **identify the location of users during emergencies**.

Examples

- Emergency call location tracking
- Disaster rescue systems

6. Location-Based Notifications

Definition

Applications send **notifications based on the user's location**.

Examples

- Promotional offers when near a store
- Event reminders when near a location

---

7. Ride-Sharing and Delivery Services

LBS are used to **match drivers, delivery agents, and customers based on their location**.

Examples

- Ride-sharing apps
- Food delivery services
- Parcel delivery tracking

Advantages of Location Based Services

1. Provides **real-time location information**.
2. Improves **navigation and travel planning**.
3. Helps businesses deliver **location-based advertisements**.
4. Enhances **safety and emergency response systems**.

Limitations of LBS

1. Requires **internet or network connectivity**.
2. Continuous GPS usage may **consume battery power**.
3. Raises **privacy concerns regarding location tracking**.

Conclusion :- Location Based Services (LBS) use technologies such as **GPS, Wi-Fi, and mobile networks** to determine device location and provide useful services. Applications such as **navigation, tracking, geofencing, location search, and emergency services** rely heavily on LBS to deliver real-time location information to users.

Android Networking API

Definition

The **Android Networking API** provides classes and libraries that allow Android applications to **communicate with remote servers over the internet**.
It enables applications to **send and receive data from web services, APIs, and remote databases**.

Networking APIs are used for tasks such as:

- Fetching data from web servers
- Uploading data
- Connecting with REST APIs
- Downloading files

Features of Android Networking API

1. Enables **communication between Android applications and servers**.
2. Supports **HTTP and HTTPS protocols**.
3. Allows **data exchange using formats like JSON and XML**.
4. Used to connect applications with **web services and cloud systems**.

Common Classes Used in Android Networking

Android provides several classes for networking operations.

Class	Purpose
URL	Represents a web address
HttpURLConnection	Establishes connection with a server
InputStream	Reads data from server
OutputStream	Sends data to server

Using HttpURLConnection

Definition : HttpURLConnection is a class used to **send HTTP requests and receive responses from a web server**.

Steps to Use Networking API

Step 1: Add Internet Permission

Applications must declare internet permission in the **AndroidManifest.xml** file.

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Step 2: Create URL Object

```
val url = URL("https://example.com")
```

Step 3: Open Connection

```
val connection = url.openConnection() as HttpURLConnection

connection.requestMethod = "GET"
```

Step 4: Read Server Response

```
val inputStream = connection.inputStream
```

Example: Simple Network Request

```
val url = URL("https://example.com")

val connection = url.openConnection() as HttpURLConnection

connection.requestMethod = "GET"

val reader = BufferedReader(InputStreamReader(connection.inputStream))

var line: String?

while (reader.readLine().also { line = it } != null)

 { println(line) }

 reader.close()
```

Data Formats Used in Networking

Android applications commonly exchange data using:

Format	Description
JSON	Lightweight data format used in APIs
XML	Structured data format
HTML	Web page format

Applications of Android Networking

1. Fetching data from **web APIs**
2. Social media integration
3. Online databases
4. Cloud services
5. File downloading and uploading

Advantages of Android Networking API

1. Enables **real-time data communication**.
2. Supports **web services and cloud applications**.
3. Allows integration with **REST APIs**.
4. Facilitates **online application features**.

Limitations

1. Requires **internet connectivity**.
2. Network requests may **take time due to latency**.
3. Must handle **security and data privacy issues**.

Conclusion : The Android Networking API enables applications to communicate with remote servers using protocols such as **HTTP and HTTPS**. By using classes like **URL and HttpURLConnection**, Android applications can retrieve and send data over the internet, making it possible to build applications that interact with web services and online resources.

Android Web API

Definition

An **Android Web API** is an interface that allows an Android application to **communicate with a remote web server over the internet**. It enables applications to **send requests and receive responses from web services**.

Web APIs allow Android apps to **retrieve, update, or send data to remote servers** using protocols such as **HTTP or HTTPS**.

Purpose of Web API in Android

1. To **connect Android applications with web servers**.
2. To **exchange data between mobile applications and cloud systems**.
3. To access **remote databases and services**.
4. To integrate external services like **maps, weather, payment systems, and social media**.

Working of Android Web API

The communication between an Android app and a web server follows these steps:

1. The **Android application sends a request** to a web server.
2. The **server processes the request**.
3. The server **sends back a response** to the application.
4. The Android app **displays the received data to the user**.

Example process:

Android App → Web API → Server → Response → Android App

Types of Web API Requests

Android Web APIs mainly use **HTTP request methods**.

Method	Purpose
GET	Retrieve data from server
POST	Send data to server
PUT	Update existing data
DELETE	Remove data

Data Formats Used in Web APIs

Web APIs usually exchange data in structured formats such as:

Format	Description
JSON	Lightweight data format commonly used in APIs
XML	Structured markup data format

Example JSON response:

```
{
  "name": "Harshal",
  "age": 20
}
```

Example of Using Web API in Android

Example Kotlin code for sending a **GET request**:

```
val url = URL("https://api.example.com/data")
val connection = url.openConnection() as HttpURLConnection
connection.requestMethod = "GET"
val inputStream = connection.inputStream
```

Applications of Android Web API

1. **Weather applications**
2. **Social media apps**
3. **Online shopping apps**
4. **Banking applications**
5. **News applications**

These apps retrieve data from **remote servers using APIs**.

Advantages of Web APIs

1. Enables **communication with remote servers**.
2. Allows applications to **access real-time data**.
3. Supports **integration with external services**.
4. Helps build **cloud-connected applications**.

Limitations

1. Requires **internet connectivity**.
2. Data transfer may depend on **network speed**.
3. Security measures are required to **protect data transmission**.

Conclusion : An **Android Web API** enables Android applications to interact with remote servers and services through HTTP requests. It allows applications to retrieve and update data using formats such as **JSON and XML**, making it essential for developing modern cloud-connected mobile applications.

Android Telephony API

Definition

The **Android Telephony API** is a set of classes and methods that allows Android applications to **access and control telecommunication services of a mobile device**.

It enables applications to interact with **cellular network information, phone calls, SMS services, and device telephony status**.

The Telephony API mainly works through the **TelephonyManager class**.

Features of Android Telephony API

1. Provides information about the **mobile network and SIM card**.
2. Allows applications to **monitor phone call states**.
3. Provides access to **device ID and network operator information**.
4. Enables **SMS and call-related functionalities**.

TelephonyManager Class

Definition

The **TelephonyManager** class is the main class used to **access telephony services and network information** in Android.

It provides methods to retrieve information such as:

- Network operator
- Device ID
- SIM status
- Call state

Accessing TelephonyManager

Example Kotlin code:

```
val telephonyManager = getSystemService(Context.TELEPHONY_SERVICE) as TelephonyManager
```

Important Methods of TelephonyManager

Method	Description
getDeviceId()	Returns device ID
getNetworkOperatorName()	Returns mobile network operator name
getSimOperatorName()	Returns SIM card operator
getCallState()	Returns current call state
getNetworkType()	Returns network type

Example: Getting Network Operator Name

```
val telephonyManager = getSystemService(Context.TELEPHONY_SERVICE) as TelephonyManager

val operatorName = telephonyManager.networkOperatorName

println(operatorName)
```

Phone Call States

The Telephony API can detect different phone call states.

Call State	Description
CALL_STATE_IDLE	No active call
CALL_STATE_RINGING	Incoming call
CALL_STATE_OFFHOOK	Call in progress

Required Permission

Applications must declare permission in the **AndroidManifest.xml** file to access telephony features.

Example:

```
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
```

Applications of Telephony API

1. **Call management applications**
2. **Network monitoring applications**
3. **SMS and communication apps**
4. **Mobile network diagnostic tools**

Advantages of Telephony API

1. Provides access to **mobile network information**.
2. Enables monitoring of **call and SIM status**.
3. Helps develop **communication-based applications**.
4. Supports **network-related features**.

Limitations

1. Requires **special permissions**.
2. Some features may be **restricted due to privacy policies**.
3. Functionality may vary depending on **device manufacturer and Android version**.

Conclusion : The **Android Telephony API** allows applications to access mobile telecommunication services and network information using the **TelephonyManager class**. It enables developers to create applications that interact with phone calls, network operators, and SIM-related data in Android devices.

Notifying the User in Android

Definition

Notification in Android is a message that informs the user about **events or updates occurring in an application**.
Notifications can appear **outside the application interface**, allowing users to receive important information even when the app is not active.

Notifications are commonly used for:

- Messages
- App updates
- Reminders
- Alerts

Android provides the **NotificationManager** class to manage notifications.

Components of Android Notifications

A notification usually contains:

1. **Small Icon** – Icon displayed in the status bar
2. **Title** – Notification heading
3. **Text Content** – Message or description
4. **Action** – Operation performed when the notification is tapped

---

Notifying the User

Definition

Notifying the user means **sending a notification message to inform the user about a specific event or action in the application**.

Example Kotlin Code

```
val builder = NotificationCompat.Builder(this, "channelId")

    .setSmallIcon(R.drawable.ic_launcher)

    .setContentTitle("New Message")

    .setContentText("You have a new notification")

    .setPriority(NotificationCompat.PRIORITY_DEFAULT)

val notificationManager = NotificationManagerCompat.from(this)

notificationManager.notify(1, builder.build())
```

Steps to Create a Notification

1. **Create a Notification Channel**
2. **Build the Notification**
3. **Display the Notification using NotificationManager**

Notification Channel

Android 8.0 and above require a **Notification Channel** to send notifications.

Example:

```
val channel = NotificationChannel(

    "channelId",

    "My Notifications",

    NotificationManager.IMPORTANCE_DEFAULT

)

val manager = getSystemService(NotificationManager::class.java)

manager.createNotificationChannel(channel)
```

Notifying with the Status Bar

Definition

The **Status Bar Notification** displays a small icon in the **status bar at the top of the screen** to inform users about events.

When the user **swipes down the status bar**, the full notification details are shown.

Features of Status Bar Notifications

1. Shows **icon in the status bar**.
2. Displays **notification message in the notification panel**.
3. Allows **user interaction with the notification**.
4. Can include **actions such as reply, open app, or dismiss**.

Example Notification Display

A status bar notification may display:

New Message

You have received a message

With an icon appearing in the **top notification bar**.

Applications of Notifications

1. Messaging applications
2. Email alerts
3. Social media updates
4. Calendar reminders
5. System alerts

Advantages of Notifications

1. Keeps users informed about important updates.
2. Allows interaction without opening the application.
3. Improves user engagement with the application.
4. Supports background communication.

Conclusion : Android notifications are used to **inform users about events occurring in an application**. Using components such as **NotificationManager and NotificationCompat**, applications can send notifications that appear in the **status bar and notification panel**, allowing users to receive important information quickly.

Vibrating the Phone in Android

Definition

Vibration in Android is a feature that allows an application to **make the device vibrate to notify or alert the user**.
It is commonly used for **notifications, alerts, reminders, and feedback for user actions**.

Android provides the **Vibrator class** to control the vibration of the device.

Vibrator Class

Definition

The **Vibrator class** is used to **access and control the vibration hardware of the Android device**.

It allows applications to start and stop vibration and set the **duration or pattern of vibration**.

Required Permission

To use vibration in an Android application, the following permission must be declared in the **AndroidManifest.xml** file.

```
<uses-permission android:name="android.permission.VIBRATE"/>
```

Accessing the Vibrator Service

Example Kotlin code:

```
val vibrator = getSystemService(Context.VIBRATOR_SERVICE) as Vibrator
```

Simple Vibration Example

The device can be made to vibrate for a specific duration.

```
vibrator.vibrate(500)
```

Explanation

Value	Description
500	Device vibrates for 500 milliseconds

Vibration with Pattern

Android also allows vibration with **custom patterns**.

Example:

```
val pattern = longArrayOf(0, 500, 200, 500)


vibrator.vibrate(pattern, -1)
```

Explanation

Value	Meaning
0	Start immediately
500	Vibrate for 500 ms
200	Pause for 200 ms
500	Vibrate again

Applications of Phone Vibration

1. **Incoming call alerts**
2. **Message notifications**
3. **Alarm reminders**
4. **Game feedback effects**
5. **Silent mode alerts**

Advantages of Vibration

1. Provides **silent notifications**.
2. Improves **user interaction and feedback**.
3. Useful in **silent environments**.
4. Enhances **accessibility for users**.

Limitations

1. Continuous vibration may **consume battery power**.
2. Some devices may have **hardware limitations**.

Conclusion : Vibration in Android is used to notify users through **physical device vibration**. By using the **Vibrator class** and declaring the **VIBRATE permission**, developers can control device vibration to provide alerts and feedback in Android applications.

Below are **clear, exam-ready answers** for the remaining **Unit-5 topics**. Each is structured so you can **memorize and reproduce in exams**.

Blinking the Lights in Android

Definition

Blinking lights in Android refers to using the **device LED light to notify users about events such as messages, calls, or alerts**.

Android notifications allow developers to **control the LED light color, duration, and blinking pattern** using the notification settings.

Using LED Light in Notifications

Android provides LED control through the **Notification class**.

The LED can be customized using:

- `setLights()` method

Syntax

setLights(color, onTime, offTime)

Example

```
val builder = NotificationCompat.Builder(this, "channelId")
    .setSmallIcon(R.drawable.ic_launcher)
    .setContentTitle("New Notification")
    .setContentText("LED Light Notification")
    .setLights(Color.RED, 1000, 1000)
```

Explanation

Parameter	Description
Color.RED	LED light color
1000	Light ON duration (ms)
1000	Light OFF duration (ms)

Applications

1. Message notifications
2. Missed call alerts
3. App event notifications
4. Silent alerts

Conclusion : Blinking lights in Android are used to **notify users through LED indicators**, which are controlled using notification settings such as `setLights()`.

Customizing Notification Services

Definition

Customizing notifications means modifying the **appearance, behavior, and functionality of notifications** to provide better interaction and user experience.

Android allows developers to customize notifications using **NotificationCompat.Builder**.

Features of Custom Notifications

1. Custom layout
2. Notification priority
3. Sound and vibration
4. Action buttons
5. Expanded notifications

Example

```
val builder = NotificationCompat.Builder(this, "channelId")
    .setSmallIcon(R.drawable.ic_launcher)
    .setContentTitle("Message")
    .setContentText("You have a new message")
```

```
    .setPriority(NotificationCompat.PRIORITY_HIGH)

    .setAutoCancel(true)
```

Notification Actions Example

```
builder.addAction(

    R.drawable.ic_reply,

    "Reply",

    pendingIntent

)
```

Advantages

1. Improves user experience
2. Provides interactive notifications
3. Allows quick actions without opening the app

Conclusion : Customized notifications allow developers to **enhance user interaction by controlling notification appearance, behavior, and actions**.

Application Development Using JSON in MySQL

Definition

JSON (JavaScript Object Notation) is a lightweight data format used to **exchange data between Android applications and databases such as MySQL through web services**.

In Android development, JSON is commonly used to **send and receive data from remote servers**.

Working Process

Android App → Web API → MySQL Database → JSON Response → Android App

---

Example JSON Data

```
{

 "id": 1,

 "name": "Harshal",

 "course": "BCA"

}
```

Parsing JSON in Android

Example Kotlin code:

```
val jsonObject = JSONObject(response)



val name = jsonObject.getString("name")

val course = jsonObject.getString("course")
```

Advantages of JSON

1. Lightweight data format
2. Easy to read and parse
3. Faster data exchange
4. Widely used in web APIs

Applications

1. Web services
2. Cloud-based applications
3. Online databases
4. Mobile applications

Conclusion : JSON is widely used in Android applications to **exchange data between the mobile application and MySQL databases via web APIs**.

Publishing Android Application

Definition

Publishing an Android application refers to the process of **distributing the application to users through platforms such as Google Play Store**.

Before publishing, the application must be **compiled, tested, signed, and packaged into an APK or AAB file**.

Steps to Publish Android Application

Step 1: Prepare the Application

- Remove bugs
- Optimize performance
- Test the application

Step 2: Generate Signed APK / AAB

In Android Studio:

Build → Generate Signed Bundle/APK

Step 3: Create Google Play Developer Account

Developers must create an account on:

Google Play Console

Step 4: Upload Application

Upload the following:

- APK / AAB file
- App description
- Screenshots
- App icon

Step 5: Publish Application

After review by Google, the application becomes **available for download on Google Play Store**.

Requirements for Publishing

1. Signed APK or AAB
2. Application icon
3. Screenshots
4. Privacy policy
5. App description

Advantages of Publishing Apps

1. Makes applications available to **millions of users**
2. Provides **global distribution**
3. Enables **revenue generation**

Conclusion : Publishing an Android application involves preparing the application, generating a signed APK, uploading it to the Google Play Store, and making it available for users worldwide.

Full Forms from the Android Syllabus

Abbreviation	Full Form
Android SDK	Android Software Development Kit
API	Application Programming Interface
APK	Android Package Kit
AAB	Android App Bundle
IDE	Integrated Development Environment
OHA	Open Handset Alliance
JVM	Java Virtual Machine
UI	User Interface
XML	eXtensible Markup Language
JSON	JavaScript Object Notation
SQL	Structured Query Language
SQLite	Structured Query Language Lite
GPS	Global Positioning System

LBS	Location Based Services
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
HTML	HyperText Markup Language
LED	Light Emitting Diode
SMS	Short Message Service
SIM	Subscriber Identity Module
MIME	Multipurpose Internet Mail Extensions
AVD	Android Virtual Device
OS	Operating System
RDBMS	Relational Database Management System
CRUD	Create, Read, Update, Delete
Wi-Fi	Wireless Fidelity
API Key	Application Programming Interface Key
CPU	Central Processing Unit
RAM	Random Access Memory
SDK Manager	Software Development Kit Manager
ADB	Android Debug Bridge

Android Component Full Forms

Abbreviation	Full Form
LBS	Location Based Services
UI	User Interface
GPS	Global Positioning System
DB	Database
LED	Light Emitting Diode
APK	Android Package Kit
API	Application Programming Interface

Networking Related Full Forms

Abbreviation	Full Form
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
URL	Uniform Resource Locator
URI	Uniform Resource Identifier
JSON	JavaScript Object Notation
XML	eXtensible Markup Language

Storage & Database Full Forms

Abbreviation	Full Form
SQL	Structured Query Language

SQLite	Structured Query Language Lite
DBMS	Database Management System
RDBMS	Relational Database Management System
CRUD	Create, Read, Update, Delete

Important Android Tools Full Forms

Abbreviation	Full Form
SDK	Software Development Kit
IDE	Integrated Development Environment
AVD	Android Virtual Device
ADB	Android Debug Bridge
JVM	Java Virtual Machine